Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Tile-Based Oyun Dünyası

PyGame ile Oyun Geliştirme – Hafta 10

Dr. Tolga BERBER

Bu Hafta Ne Öğreneceğiz?

1. Tile Sistemi Temelleri
2. Harita Oluşturma
3. Kamera ve Scrolling
4. Seviye Tasarımı

Tile Sistemi Temelleri

Tile Nedir?

- Haritanın en küçük yapı birimi
- Sabit boyutlu kare (ör. 32×32)
- Her tile bir **ID** ile temsil edilir
- Harita = 2B liste (int matrisi)

Neden Tile?

Bellek verimli
Kolay düzenleme
Hızlı çarpışma
Tekrar kullanılabilir

2B Liste ile Harita

```
# 0=bos, 1=cim, 2=tas
HARITA = [
    [2, 2, 2, 2, 2],
    [2, 1, 1, 1, 2],
    [2, 1, 0, 1, 2],
    [2, 1, 1, 1, 2],
    [2, 2, 2, 2, 2],
]
# Erisim: HARITA[row][col]
print(HARITA[2][2])  # 0
```

Sıra Önemli!

`HARITA[row][col]` doğru — önce satır, sonra sütun.

Tileset ve subsurface()

```
tileset = pygame.image.load("tileset.png").convert_alpha()

TILE = 32
tiles = []
for row in range(2):
    for col in range(4):
        rect = pygame.Rect(col*TILE, row*TILE, TILE, TILE)
        tiles.append(tileset.subsurface(rect))

# Kullanim: tiles[tile_id]
ekran.blit(tiles[2], (100, 100))
```

subsurface() = Referans

Yeni kopya değil, orijinalin bir bölümüne referans verir. Çok hızlı.

Grid ↔ Piksel Dönüşümü

Piksel → Grid

```
col = px // TILE
row = py // TILE
```

Grid → Piksel

```
x = col * TILE
y = row * TILE
```

Sınır Kontrolü

```
if 0 <= row < len(harita) and 0 <= col < len(harita[0]):
```

Örnek

`piksel (150, 75) → grid (4, 2)`

Harita Oluşturma

CSV Harita Yükleme

```python
import csv
from pathlib import Path

def harita_yukle(yol):
    with open(yol, newline="") as f:
        return [[int(x) for x in row]
                for row in csv.reader(f) if row]

yol = Path(__file__).parent / "harita.csv"
harita = harita_yukle(yol)
```

CSV= Düz Metin

```
2,2,2,2,2
2,1,1,1,2
2,2,2,2,2
```

Tiled Editor

- Ücretsiz tile editörü
- `https://www.mapeditor.org`
- Fareyle tile yerleştir
- Çok katmanlı destek
- `.tmx` (XML) kaydeder

Tipik Katmanlar

arkaplan
platform (solid)
esyalar
onplan

pytmx ile .tmx Yükleme

```python
from pytmx.util_pygame import load_pygame

tmx = load_pygame("harita.tmx")

print(tmx.width, tmx.height)          # grid boyutu
print(tmx.tilewidth, tmx.tileheight) # tile piksel

# Tum gorunur katmanlari ciz
for layer in tmx.visible_layers:
    if hasattr(layer, "tiles"):
        for x, y, image in layer.tiles():
            ekran.blit(image, (x*TILE, y*TILE))
```

Kurulum

```
uv add pytmx
```

CSV vs pytmx Özeti

CSV

- + Ek bağımlılık yok
- + Hızlı prototip
- – Tek katman
- – Elle zor

Tiled + pytmx

- + Görsel editor
- + Çok katmanlı
- + Obje grupları
- – Kurulum gerekir

Kamera ve Scrolling

World vs Screen Space

World-Space

Nesnenin dünyadaki sabit konumu
(`rect.x = 1500`)

Screen-Space

Ekrandaki görünen konumu
(`rect.x - offset.x`)

Temel Formül

```
ekran = world - offset
```

Camera Sınıfı

```
class Camera:
    def __init__(self, gen, yuk, lerp=0.1):
        self.offset = pygame.Vector2(0, 0)
        self.gen, self.yuk = gen, yuk
        self.lerp = lerp

    def apply(self, rect):
        return rect.move(-int(self.offset.x),
                         -int(self.offset.y))

    def update(self, hedef):
        istenen = pygame.Vector2(
            hedef.centerx - self.gen // 2,
            hedef.centery - self.yuk // 2)
        self.offset += (istenen - self.offset) * self.lerp
```

Lerp (Yumuşak Takip)

Formül

```
yeni = eski + (hedef - eski) * t
```

- $t = 0.02$: Çok yumuşak, lazy
- $t = 0.1$: Dengeli, doğal (varsayılan)
- $t = 0.5$: Sert, aksiyonel
- $t = 1.0$: Anlık (lerp yok)

Dikkat

$t > 1 =$ overshoot, titreme. $t < 0 =$ geri kayma.

Kamera Clamp

```
def _clamp(self):
    max_x = max(0, self.dunya_gen - self.ekran_gen)
    max_y = max(0, self.dunya_yuk - self.ekran_yuk)
    if self.offset.x < 0:      self.offset.x = 0
    if self.offset.y < 0:      self.offset.y = 0
    if self.offset.x > max_x:  self.offset.x = max_x
    if self.offset.y > max_y:  self.offset.y = max_y
```

Niçin max(0, ...)?

Harita ekrandan küçükse sonuç negatif olur → kamera geri kayar.

Seviye Tasarımı

Tile Gruplarına Ayırma

```
def tile_gruplari(harita):
    katilar, tehlikeler, altinlar = [], [], []
    for r, satir in enumerate(harita):
        for c, tid in enumerate(satir):
            rect = pygame.Rect(c*TILE, r*TILE, TILE, TILE)
            if tid == 2:    katilar.append(rect)
            elif tid == 5:  tehlikeler.append(rect)
            elif tid == 4:  altinlar.append(rect)
    return katilar, tehlikeler, altinlar
```

Grup Başına Davranış

Solid = durma | Tehlike = zarar | Altın = skor

Platform Çarpışması: Yatay + Dikey Ayır

```
# 1. YATAY hareket + carpisma
self.rect.x += self.vx
for k in katilar:
    if self.rect.colliderect(k):
        if self.vx > 0:   self.rect.right = k.left
        elif self.vx < 0: self.rect.left = k.right

# 2. DIKEY hareket + carpisma
self.vy += 0.5
self.rect.y += int(self.vy)
for k in katilar:
    if self.rect.colliderect(k):
        if self.vy > 0:
            self.rect.bottom = k.top
            self.yerde = True
        elif self.vy < 0:
            self.rect.top = k.bottom
```

Tehlike ve Spawn

```
class Oyuncu:
    def __init__(self, x, y):
        self.spawn = (x, y)
        self.rect = pygame.Rect(x, y, 24, 32)
        self.can = 3

    def yeniden_dogur(self):
        self.rect.topleft = self.spawn
        self.vx = self.vy = 0

# Ana dongu
for t in tehlikeler:
    if oyuncu.rect.colliderect(t):
        oyuncu.can -= 1
        oyuncu.yeniden_dogur()
        break  # cift zarar onle
```

Toplanabilir: Liste Kopyası

```
skor = 0

# GUVENLI silme: kopyasi uzerinde dolas
for a in altinlar[:]:
    if oyuncu.rect.colliderect(a):
        altinlar.remove(a)
        skor += 10
```

Yanlış: altinlar Üzerinde Silme

İterasyon sırasında listeyi değiştirmek indeks kaydırır. Her zaman `[:]` kopyası kullan.

Mini Proje: Zıplayan Macera Seviye 1

Proje Bileşenleri

- `Seviye` sınıfı: CSV yükle, gruplara ayır
- `Camera`: lerp + clamp
- `Oyuncu`: yerçekimi, zıplama, çarpışma
- **katilar** → platformlar
- **tehlikeler** → diken
- **altinlar** → toplanabilir
- HUD: Can + Skor

Seviye Tasarım Kuralları

Başlangıç güvenli | İlk zıplama kolay | Risk-ödül dengesi

Özet

Bu Hafta Öğrendiklerimiz

- Tile sistemi ve 2B harita
- Tileset + subsurface()
- CSV ve Tiled (`.tmx`)
- pytmx ile yükleme
- World/screen space
- Camera sınıfı + lerp + clamp
- Platform çarpışması
- Tehlike, toplanabilir

Gelecek Hafta

- Paralaks arkaplan
- Animasyonlu tile'lar
- Kamera shake
- Seviye geçişleri